

## Practical Week 10

### Aim

To investigate Linux inode structures using `ls -li`, `stat`, and `find`, and understand the difference between hard links and symbolic links.

```
saikrishna__90156@DESKTOP-2JF9H5T: X + v
saikrishna__90156@DESKTOP-2JF9H5T:~$ mkdir -p ~/practical_week10
saikrishna__90156@DESKTOP-2JF9H5T:~$ cd ~/practical_week10
saikrishna__90156@DESKTOP-2JF9H5T:~/practical_week10$ echo "Linux inode practical" > original.txt
saikrishna__90156@DESKTOP-2JF9H5T:~/practical_week10$ cat original.txt
Linux inode practical
saikrishna__90156@DESKTOP-2JF9H5T:~/practical_week10$ ls -li original.txt
13688 original.txt
saikrishna__90156@DESKTOP-2JF9H5T:~/practical_week10$ stat original.txt
  File: original.txt
  Size: 22          Blocks: 8          IO Block: 4096   regular file
Device: 8,48      Inode: 13688
Access: (0644/-rw-r--r--)  Uid: ( 1000/navaneeth)   Gid: ( 1000/navaneeth)
Access: 2026-09-27 10:45:49.332245680 +0000
Modify: 2026-09-27 10:45:43.920245126 +0000
Change: 2026-09-27 10:45:43.920245126 +0000
 Birth: 2026-09-27 10:45:43.920245126 +0000
saikrishna__90156@DESKTOP-2JF9H5T:~/practical_week10$ ln original.txt hardlink.txt
saikrishna__90156@DESKTOP-2JF9H5T:~/practical_week10$ ls -li original.txt hardlink.txt
13688 hardlink.txt 13688 original.txt
saikrishna__90156@DESKTOP-2JF9H5T:~/practical_week10$ stat original.txt
  File: original.txt
  Size: 22          Blocks: 8          IO Block: 4096   regular file
Device: 8,48      Inode: 13688
Access: (0644/-rw-r--r--)  Uid: ( 1000/navaneeth)   Gid: ( 1000/navaneeth)
Access: 2026-09-27 10:45:49.332245680 +0000
Modify: 2026-09-27 10:45:43.920245126 +0000
Change: 2026-09-27 10:46:13.084245567 +0000
 Birth: 2026-09-27 10:45:43.920245126 +0000
saikrishna__90156@DESKTOP-2JF9H5T:~/practical_week10$ echo "Data added through hard link" >> hardlink.txt
saikrishna__90156@DESKTOP-2JF9H5T:~/practical_week10$ cat original.txt
Linux inode practical
Data added through hard link
```

### Observation

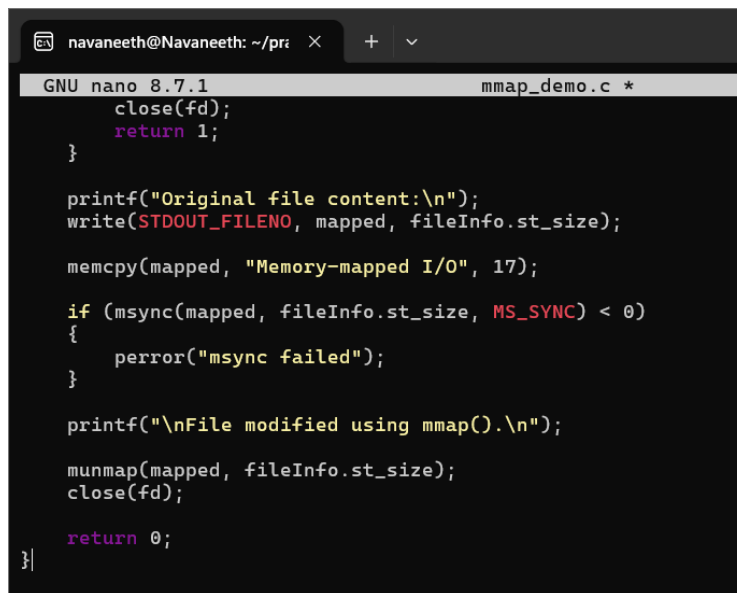
- original.txt and hardlink.txt have the **same inode number**.
- The hard link increases the link count.
- symlink.txt has a different inode and points to original.txt.
- `find -inum` can locate directory entries associated with an inode.

### Result

Linux inode information was successfully examined, and hard links and symbolic links were created and verified using Linux commands.

## Aim

To use the Linux `mmap()` system call for reading and writing a file and understand its difference from traditional `read()` and `write()` operations.



```
navaneeth@Navaneeth: ~/pr: × + ▾
GNU nano 8.7.1 mmap_demo.c *
    close(fd);
    return 1;
}

printf("Original file content:\n");
write(STDOUT_FILENO, mapped, fileInfo.st_size);

memcpy(mapped, "Memory-mapped I/O", 17);

if (msync(mapped, fileInfo.st_size, MS_SYNC) < 0)
{
    perror("msync failed");
}

printf("\nFile modified using mmap().\n");

munmap(mapped, fileInfo.st_size);
close(fd);

return 0;
}|
```



```
GNU nano 8.7.1 traditional_io.c *
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main()
{
    int fd;
    char buffer[1024];
    ssize_t bytesRead;

    fd = open("mmap_input.txt", O_RDONLY);

    if (fd < 0)
    {
        perror("Error opening file");
        return 1;
    }

    bytesRead = read(fd, buffer, sizeof(buffer) - 1);

    if (bytesRead < 0)
    {
        perror("Error reading file");
        close(fd);
        return 1;
    }

    buffer[bytesRead] = '\0';

    printf("File content using read():\n");
    printf("%s\n", buffer);

    close(fd);

    return 0;
}
```

^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute ^C Location  
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^\_ Go To Line

## Observation

The file was successfully mapped into memory using `mmap()`. The program accessed and modified the mapped memory, and the changes were synchronized back to the file using `msync()`. A traditional `read()` program was also implemented for comparison.

## Result

**File reading and writing using `mmap()` was successfully implemented and compared with traditional `read()/write()` file I/O.**